

(More on) Linear Models

DS 4440 · LECTURE 2

1. Last time: the perceptron

Our model predicted a label by thresholding a linear score:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x}_i > 0, \\ -1 & \text{otherwise.} \end{cases}$$

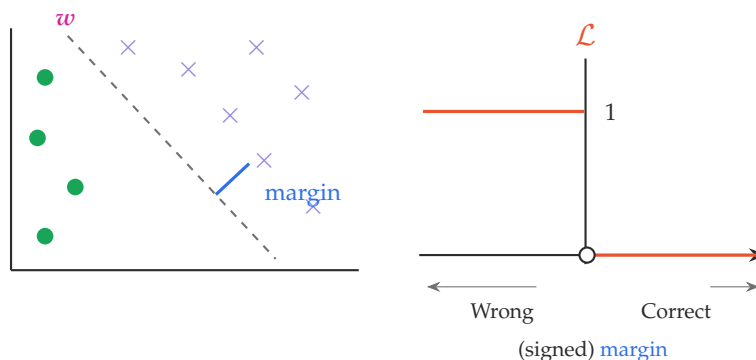
★ As before, the bias terms are folded into $\mathbf{x}_i$ and $\mathbf{w}$, which is why the threshold is 0 rather than some τ .

Given $\langle \mathbf{X}, \mathbf{y} \rangle$ we **fit** this model: we find a $\hat{\mathbf{w}}$ that *minimizes a loss*. For the perceptron that loss was

$$\mathcal{L}(\hat{y}_i, y_i) = \begin{cases} 0 & \text{if } \hat{y}_i = y_i, \\ 1 & \text{otherwise.} \end{cases}$$

This is the **0/1 loss**. It is simple, and it is exactly what we care about at evaluation time — but it is not a great thing to optimize. *Why?*

2. Margins, and why 0/1 loss is awkward

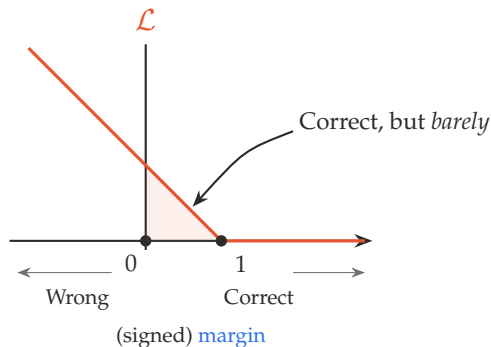


The useful quantity here is the **(signed) margin** $y_i (\mathbf{w} \cdot \mathbf{x}_i)$: how far an example sits from the decision boundary, signed so that positive means *correct* and negative means *wrong*. Plotted against the margin, the 0/1 loss is a step: it charges 1 everywhere on the wrong side and 0 everywhere on the right side.

Addition (not in the original notes): this is the “why”. The step is flat on both sides, so its derivative is 0 almost everywhere and undefined at the jump — a gradient-based optimizer gets no signal from it. It also treats a barely-correct point and a confidently-correct point identically, so nothing pushes the boundary away from the data. Minimizing it exactly over a linear model is NP-hard in general.

3. One alternative: hinge loss

$$\mathcal{L}_H(\hat{w}x_i, y_i) \stackrel{\text{def}}{=} \max\{0, 1 - y_i(\hat{w}x_i)\}$$



The hinge keeps charging until the margin exceeds 1, so points that are correct but close to the boundary still contribute loss and still produce a gradient. Past a margin of 1 the loss is flat and those examples stop influencing the fit.

Addition: the hinge is convex and is an upper bound on the 0/1 loss, which is why minimizing it is a sensible surrogate. It is not differentiable at margin = 1; in practice we take a subgradient there (pick either side).

But hinge is just one option — there are **many loss functions** we can use.

4. In general

Whatever loss we pick, fitting means the same thing:

$$\begin{aligned}\hat{w} &\leftarrow \arg \min_w \mathcal{L}(X, y | w) \\ &= \arg \min_w \sum_{i=1}^N \mathcal{L}(x_i, y_i | w).\end{aligned}$$

Regularization

Only minimizing loss may result in **overfitting**. So we often add **regularization** on the parameters, e.g.

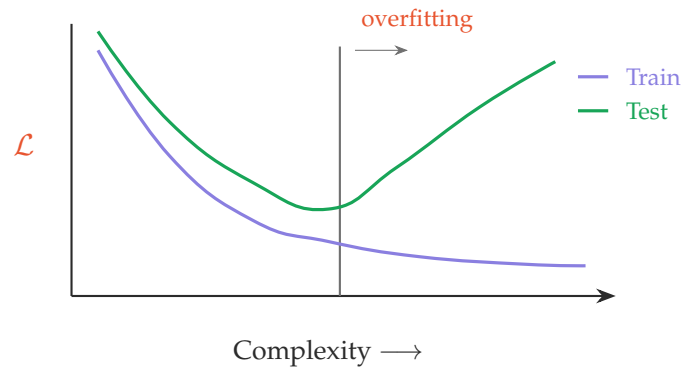
$$R(w) \stackrel{\text{def}}{=} \|w\|_2^2 = \sum_{j=1}^D w_j^2.$$

The **composite objective** is then:

$$\arg \min_w \underbrace{\mathcal{L}(X, y | w)}_{\text{empirical loss}} + \underbrace{\lambda R(w)}_{\text{regularizer}}$$

λ is a **hyperparameter** that dictates regularization “strength”:

$$\uparrow \lambda \text{ (bias)} \longrightarrow \text{lower variance.}$$

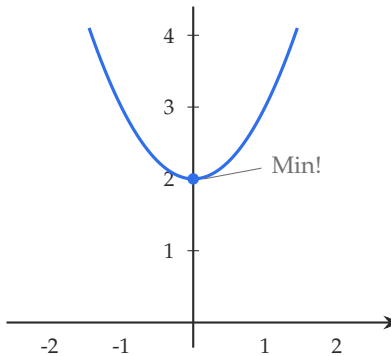


5. Central question: how do we find $\hat{w}$?

How do we find $\hat{w}$?

We did something kind of *ad hoc* for the perceptron. A more general strategy — and the workhorse of **deep learning** — is **gradient-based optimization**.

A quick refresher



For $f(x) = x^2 + 2$, the **minimum** value is where

$$\frac{d}{dx}f(x) = 0, \quad \text{and} \quad \frac{d}{dx}f(x) = 2x \implies \text{minimum at } x = 0.$$

Derivative rules

$$\frac{d}{dx} c = 0$$

$$\frac{d}{dx} x \cdot c = c$$

$$\frac{d}{dx} \ln x = \frac{1}{x}$$

$$\frac{d}{dx} x^k = k \cdot x^{(k-1)}$$

power rule

$$\frac{d}{dx} [f(x) + g(x)] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

sum rule

$$\frac{d}{dx} f(x) \cdot g(x) = \left(\frac{d}{dx} f(x) \right) \cdot g(x) + f(x) \cdot \left(\frac{d}{dx} g(x) \right)$$

product rule

$$\frac{d}{dx} f(\underbrace{g(x)}_u) = \frac{d}{du} f(u) \cdot \frac{d}{dx} u$$

chain rule

6. Gradients

For a vector of parameters $\vec{w} = [w_1 \cdots w_d]$ and a scalar-valued $f(\vec{w})$, the **gradient** collects the partial derivatives:

$$\nabla_{\vec{w}} f(\vec{w}) = \left[\frac{\partial f}{\partial w_1} \cdots \frac{\partial f}{\partial w_d} \right].$$

For example, with

$$\begin{aligned} f(\vec{w}) &= w_1^3 \cdot w_2 + 5w_2 + w_1 \cdot w_3, \\ \nabla_{\vec{w}} f(\vec{w}) &= [3w_1^2 w_2 + w_3, \quad w_1^3 + 5, \quad w_1]. \end{aligned}$$

If f maps to m outputs, we have a **Jacobian**: one gradient per output, stacked into a matrix.

$$\underbrace{\begin{bmatrix} \frac{\partial}{\partial x_1} f_1(x) & \cdots & \frac{\partial}{\partial x_d} f_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(x) & \cdots & \frac{\partial}{\partial x_d} f_m(x) \end{bmatrix}}_{d \text{ columns}} \left. \vphantom{\begin{bmatrix} \frac{\partial}{\partial x_1} f_1(x) & \cdots & \frac{\partial}{\partial x_d} f_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(x) & \cdots & \frac{\partial}{\partial x_d} f_m(x) \end{bmatrix}} \right\} m \text{ rows}$$

Each row is $\nabla_x f_i(x)$, a d -dimensional gradient.

7. Useful identities

$$\nabla_x A x = A$$

Let $A \in \mathbb{R}^{n \times m}$ and $x \in \mathbb{R}^{m \times 1}$. Row i of the product is $A_{i1}x_1 + \cdots + A_{im}x_m$, so

$$(\nabla_x A x)_i = \nabla_x (A_{i1}x_1 + \cdots + A_{im}x_m) = [A_{i1} \cdots A_{im}],$$

which is exactly row i of A . Stacking over i :

$$\nabla_x A x = A$$

$$\nabla_x x A = A^\top$$

Now let $A \in \mathbb{R}^{n \times m}$ and $x \in \mathbb{R}^{1 \times n}$. Column j of the product is $x_1 A_{1j} + \cdots + x_n A_{nj}$, so

$$(\nabla_x x A)_j = \nabla_x (x_1 A_{1j} + \cdots + x_n A_{nj}) = [A_{1j} \ A_{2j} \ \cdots \ A_{nj}],$$

the j -th column of A laid out as a row. Stacking over j :

$$\nabla_x x A = A^\top$$

Elementwise operations

Suppose λ is applied elementwise to x :

$$y = \lambda \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} = \begin{bmatrix} \lambda(x_1) \\ \lambda(x_2) \\ \vdots \\ \lambda(x_m) \end{bmatrix}, \quad \nabla_x y = ?$$

Output i depends only on input i , so every off-diagonal partial vanishes and the Jacobian is **diagonal**:

$$\nabla_{\mathbf{x}} \mathbf{y} = \begin{bmatrix} \frac{\partial}{\partial x_1} \lambda(x_1) & \cdots & \frac{\partial}{\partial x_m} \lambda(x_1) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} \lambda(x_m) & \cdots & \frac{\partial}{\partial x_m} \lambda(x_m) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} \lambda(x_1) & & 0 \\ & \ddots & \\ 0 & & \frac{\partial}{\partial x_m} \lambda(x_m) \end{bmatrix}$$

8. Exercise: gradient of hinge loss + ℓ_2 regularizer

Revisiting the start of today, let's take the gradient of hinge loss plus an ℓ_2 regularizer. Recall

$$\mathcal{L}_H(\hat{\mathbf{w}}\mathbf{x}_i, y_i) \stackrel{\text{def}}{=} \max\{0, 1 - \underline{y_i(\hat{\mathbf{w}}\mathbf{x}_i)}\}.$$

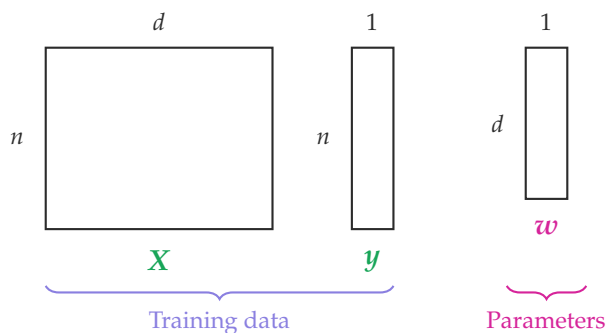
Assume for now that the underlined term is the max (i.e. the example is inside the margin), and differentiate:

$$\begin{aligned} \nabla_{\mathbf{w}} \left[(1 - y_i(\hat{\mathbf{w}}\mathbf{x}_i)) + \lambda \frac{1}{2} \sum_j w_j^2 \right] &= \nabla_{\mathbf{w}} (-y_i(\hat{\mathbf{w}}\mathbf{x}_i)) + \frac{\lambda}{2} \nabla_{\mathbf{w}} \sum_j w_j^2 \\ &= -y_i\mathbf{x}_i + \lambda\mathbf{w}. \end{aligned}$$

The constant 1 differentiates away. For the regularizer, $\partial w_j^2 / \partial w_k = 0$ for $k \neq j$ and $2w_j$ when $j = k$, so the sum contributes $2\mathbf{w}$, and the $\frac{1}{2}$ out front cancels it to $\lambda\mathbf{w}$.

The data term $-y_i\mathbf{x}_i$ drops if 0 is the max — that is, once the example is classified with margin at least 1, only the regularizer pulls on $\mathbf{w}$.

9. Least squares regression, from an ML view



The loss is squared error:

$$\mathcal{L} = \sum_{i=1}^n (y_i - \underbrace{\mathbf{x}_i \mathbf{w}}_{\hat{y}_i})^2 = \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2.$$

To differentiate, apply the chain rule with $\mathbf{u} = \mathbf{y} - \mathbf{X}\mathbf{w}$:

$$\nabla_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 = \underbrace{\left(\frac{\partial \mathbf{u}}{\partial \mathbf{w}} \right)^\top}_{\text{Jacobian}} \cdot \underbrace{\nabla_{\mathbf{u}} \|\mathbf{u}\|^2}_{\text{incoming grad}}$$

The Jacobian of $\mathbf{u}$ with respect to $\mathbf{w}$ is $-\mathbf{X}$ (by the first identity above), and the incoming gradient is $2\mathbf{u}$, so

$$\begin{aligned}\nabla_{\mathbf{w}} \mathcal{L} &= -\mathbf{X}^\top \cdot 2 \cdot \mathbf{u} = -\underset{d \times n}{2\mathbf{X}^\top} (\underset{n \times 1}{\mathbf{y} - \mathbf{X}\mathbf{w}}) \\ &= -\underset{d \times 1}{2\mathbf{X}^\top \mathbf{y}} + \underset{d \times 1}{2\mathbf{X}^\top \mathbf{X} \mathbf{w}}.\end{aligned}$$

Set to $\vec{0}$:

$$\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y} \quad \implies \quad \mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

For linear regression we can solve **analytically**. But that is not always the case!

Addition: this requires $\mathbf{X}^\top \mathbf{X}$ to be invertible, which fails when $d > n$ or when features are collinear. Adding an ℓ_2 regularizer fixes it — $\mathbf{w}^ = (\mathbf{X}^\top \mathbf{X} + \lambda I)^{-1} \mathbf{X}^\top \mathbf{y}$ — which is ridge regression.*

10. Gradient descent

A more flexible approach is **gradient descent**, an iterative optimization algorithm.

$$\hat{\mathbf{w}}^{(t+1)} \leftarrow \hat{\mathbf{w}}^{(t)} - \underbrace{\alpha}_{\text{learning rate}} \nabla_{\mathbf{w}} \mathcal{L}(\underbrace{\mathbf{X}, \mathbf{y}}_{\text{usually batched}} \mid \hat{\mathbf{w}}^{(t)})$$

Take the gradient, step downhill by α , repeat. No closed form required — which is why this, and not the normal equations, is what scales to deep networks.

Let's see it in Colab!